
Software Requirements Specification

for

FARM MANAGER

Version 1.0

Prepared by SIBIRAJAN A

COIMBATORE INSTITUTE OF TECHNOLOGY

05.10.2025

Table of Contents

Table of Contents	ii
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	2
1.5 References.....	2
2. Overall Description	2
2.1 Product Perspective.....	2
2.2 Product Functions	2
2.3 User Classes and Characteristics	3
2.4 Operating Environment.....	3
2.5 Design and Implementation Constraints	3
2.6 User Documentation	3
2.7 Assumptions and Dependencies	4
3. External Interface Requirements	4
3.1 User Interfaces	4
3.2 Hardware Interfaces	4
3.3 Software Interfaces	4
3.4 Communications Interfaces	4
4. System Features	5
4.1 System Feature 1	5
4.2 System Feature 2.....	5
4.3 System Feature 3.....	6
4.4 System Feature 4.....	6
4.5 System Feature 5.....	6
5. Other Nonfunctional Requirements.....	18
5.1 Performance Requirements	18
5.2 Safety Requirements	18
5.3 Security Requirements	18
5.4 Software Quality Attributes	18
5.5 Business Rules	19
6. Other Requirements	19
Appendix A: Glossary.....	19
Appendix B: Analysis Models.....	19
Appendix C: To Be Determined List.....	19

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the requirements for a web-based *Farming Expense Notes Management System*, Version 1.0. This application is designed to help farmers, especially those with limited technical or financial calculation skills, systematically record, track, and manage their daily farming expenses such as labour wages, fertilizer costs, and other spending. The system provides simple note-taking with date, topic, description, and price details, auto-calculates totals for each note, and includes alerts and reminders through future markings and notifications. Basic user authentication using a simple code is provided to secure access. The application is designed to be simple, clear, and user-friendly to ensure that farmers can easily manage their expense records and receive timely reminders, supporting better farm management and financial planning.

1.2 Document Conventions

This document follows standard IEEE SRS guidelines. All requirements are described in plain language with consistent formatting.

- Requirements are described in a simple, understandable format.
- Dates will follow the Day DD Month YYYY format, for example, Wednesday 03 July 2025.
- Headings and subheadings follow numbering consistent with IEEE standard sections.

1.3 Intended Audience and Reading Suggestions

This document is primarily intended for:

- The developer, who will design, implement, test, and deploy the entire web application.
- The project supervisor or academic evaluator who will review this micro project for assessment.
- Farmers and end users who may review this document for feedback on usability, clarity, and relevance to their needs.

Readers are encouraged to start with Section 1 for an overview, then proceed to Section 2 for a broader product description, and finally review Sections 3 and 4 for detailed functional and interface requirements.

1.4 Product Scope

The Farming Expense Notes Management Application is a web-based solution that allows farmers to easily record and manage farming expenses in a structured manner. It provides functionality to create, view, and delete notes, store details like date, topic, optional descriptions, and prices, and to calculate total expenses. Additionally, it supports future alert markings for important reminders through notifications. The software targets to improve record-keeping for farmers in a simple, user-friendly, and language-accessible way. Ultimately, this tool aims to promote better financial tracking and planning in the farming community.

1.5 References

- IEEE Software Requirements Specification (SRS) Template
- HTML/CSS and Python Flask framework documentation (planned technology stack)
- [MDN Web Docs](#) for web standards
- Python Flask documentation

2. Overall Description

2.1 Product Perspective

This product is a stand-alone web application intended primarily for small and medium-scale farmers to manage and track expenses efficiently. It builds upon the tradition of manual record-keeping, substituting with a systematic, digital approach tailored for users with minimal technical background. The product uses industry-standard web technologies (Flask, HTML, CSS) and is designed for easy deployment and maintenance.

2.2 Product Functions

- ✓ Simple user authentication and registration
- ✓ Creation, viewing, editing, and deletion of expense notes
- ✓ Input of essential details: date, topic, description, and price

- ✓ Automatic calculation and display of total expenses per note
- ✓ Setting of reminders and notifications for future events or expenses
- ✓ Viewing of notifications and reminders

2.3 User Classes and Characteristics

- ✓ **Farmers:** Primary users with limited technical expertise; require clear and intuitive workflows
- ✓ **Evaluator/Supervisor:** Reviews the system for assessment and feedback; possesses higher technical expertise
- ✓ **Developer:** Configures, manages, and maintains the application

2.4 Operating Environment

- ✓ **Cross-platform operation:** Accessible on any device with a modern web browser (desktop, laptop, or mobile)
- ✓ **Backend:** Built using Python 3.x with Flask framework
- ✓ **Database:** Uses SQLite or a compatible database system
- ✓ **Deployment:** Can be hosted locally, on a LAN, or any standard web server supporting Python applications

2.5 Design and Implementation Constraints

- ✓ **User interface:** Must remain simple and free from technical jargon
- ✓ **Data storage:** Utilizes a common, lightweight database for persistent storage
- ✓ **Security:** Ensures secure handling of user-provided credentials for authentication
- ✓ **Compliance:** Adheres to basic web accessibility and usability standards

2.6 User Documentation

- ✓ **In-app assistance:** Includes tooltips and instructional labels for user guidance
- ✓ **User manual:** Provides a simple PDF/HTML manual with step-by-step instructions and screenshots

- ✓ **FAQ section:** Contains answers to frequently asked questions and common issues

2.7 Assumptions and Dependencies

- ✓ **Device access:** Users have at least one device with network connectivity and a modern web browser
- ✓ **Data management:** User data is stored and retrieved locally unless the application is server-deployed
- ✓ **Technology dependency:** Project relies on ongoing support for Flask and associated web technologies

3. External Interface Requirements

3.1 User Interfaces

- **Responsive design:** Mobile-friendly web interface adaptable to various screen sizes
- **Core screens:** Includes login, registration, note management, and notification pages
- **UI elements:** Features clear buttons, error/success alerts, and date pickers for better usability

3.2 Hardware Interfaces

- Runs on standard computing hardware, no special device requirement
- Recommend a minimum of 2 GB RAM and a recent web browser

3.3 Software Interfaces

- Integration with SQLite database via Flask ORM/SQLAlchemy
- Compatibility with modern web browsers (Chrome, Firefox, Edge)
- No third-party API dependency

3.4 Communications Interfaces

- Uses HTTP(S) for client-server data transfer

- Local deployments may not require network communication; server deployments use standard web protocols
- No requirement for email, only in-app notifications

4. System Features

Feature	Description & Priority	Related Requirements
User Authentication	Secure login/register with code; High	REQ-1: User Login, REQ-2: Register
Note Management	Create/view/edit/delete expense notes; High	REQ-3: CRUD operations
Expense Calculation	Auto-calculation of total expenses; High	REQ-4: Total Calculation
Reminders/Notifications	Set/view date-based alerts; Medium	REQ-5: Reminder Management
Data Validation	Input checks for dates and prices; High	REQ-6: Validation

4.1 System Feature: User Authentication Module

Purpose: Provides secure login and access control for registered users to protect personal data and system integrity.

Key Functions:

- Validates user credentials during login.
- Allows multiple retry attempts on authentication failure.
- Supports secure logout to terminate user session and prevent unauthorized access.

4.2 System Feature: Note Management Module

Purpose:

Enables users to efficiently create, edit, view, and delete farming expense notes with essential details.

Key Functions:

- User-friendly interface for inputting date, topic, description, and price.
- Facilitates modification and removal of existing records.

- Displays organized list of notes for quick navigation and reference.

4.3 System Feature: Expense Calculation Module

Purpose:

Automatically calculates total expenses from entered note items to assist users in financial tracking.

Key Functions:

- Sums prices in real-time as entries are added or updated.
- Handles invalid or incorrect data entries gracefully.
- Updates totals dynamically without manual intervention.

4.4 Reminder and Notification Module

Purpose:

Helps users set and receive alerts for future farming expense events and important reminders.

Key Functions:

- Allows creation and storage of date-based reminders with optional notes.
- Triggers in-app notifications when reminder dates occur.
- Provides an interface to view current and past notifications.

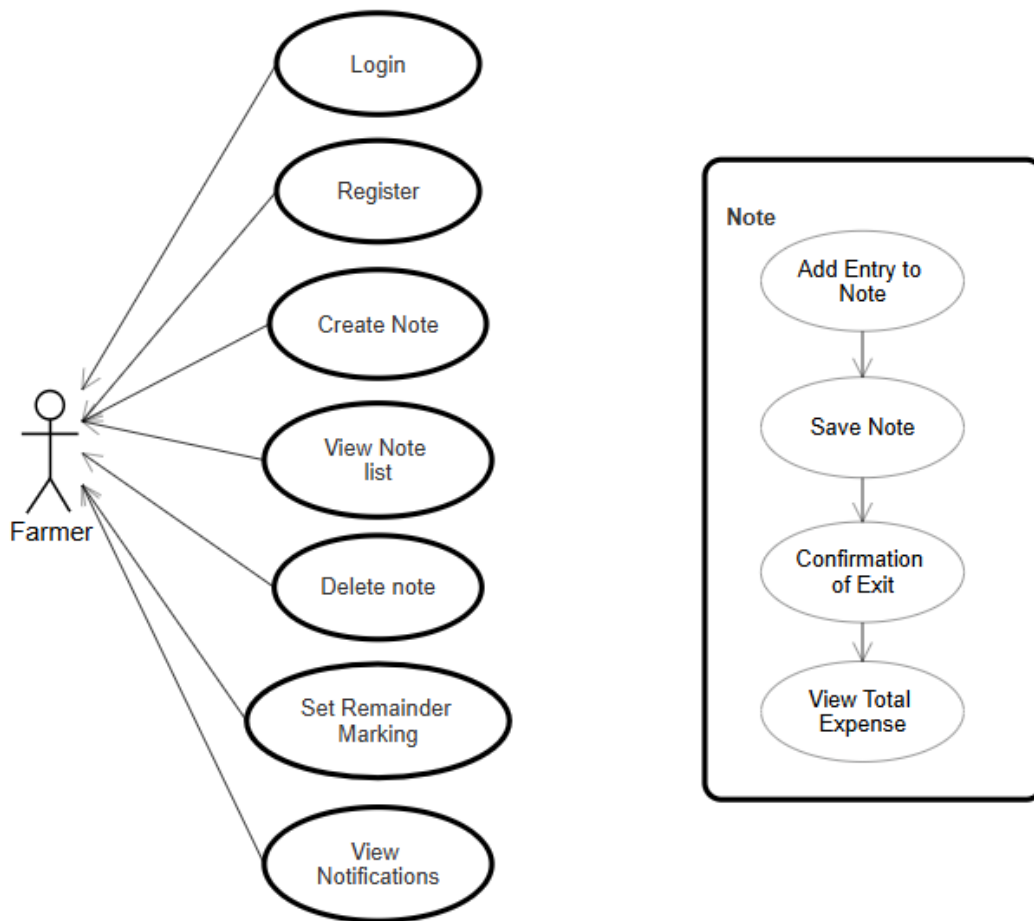
4.5 System Feature: Data Validation Module

Purpose:

Ensures input data correctness and consistency across the application to maintain reliability.

Key Functions:

- Validates formats for dates, prices, and mandatory fields.
- Provides auto-correction or prompts user warnings for invalid data.
- Prevents saving of incomplete or erroneous information.

USE CASE DIAGRAM:**Description****UC1: Login to Application**

Primary Actor: User (Farmer)

Secondary Actor: Authentication Module (System)

Preconditions:

→ User must have already registered (i.e., code exists)

Trigger:

→ User opens the application

Main Success Scenario:

1. User enters code

2. System checks the code in database
3. If matched, user is taken to the Home Page
4. System loads user profile data and settings

Exception Scenarios:

1. If code is incorrect
 - System shows error and allows retry
2. If user tries to login without code
 - System prompts for valid code

UC2: Register New User

Primary Actor: New User (Farmer)

Secondary Actor: Registration Module

Preconditions:

- User is not yet registered

Trigger:

- User clicks on “Create Profile”

Main Success Scenario:

1. User enters name and unique code
2. System validates input
3. System stores new user in database
4. Redirects to login page

Exception Scenarios:

1. Code already exists
 - System prompts to choose another code

UC3: Create a Note

Primary Actor: User

Secondary Actor: Notes Module

Preconditions:

- User is logged in

Trigger:

- User clicks on “Create Note”

Main Success Scenario:

1. System shows note creation interface
2. User adds entries with date, topic, optional description, price
3. System formats date to “Day Date Month Year”
4. User clicks “Save”
5. System stores note to DB

Exception Scenarios:

1. User exits without saving
 - System shows “Save changes?” dialog

2. Date or price is invalid
→ System autocorrects or shows warning

UC4: View Note List

Primary Actor: User

Preconditions:

- User has created one or more notes

Trigger:

- User clicks on “Notes”

Main Success Scenario:

1. System displays all notes with headings
2. User selects a note to view/edit

UC5: Delete Note

Primary Actor: User

Preconditions:

- Notes exist

Trigger:

- User clicks on delete icon for a note

Main Success Scenario:

1. System prompts confirmation
2. On confirm, deletes the note
3. Updates note list

UC6: Update Entries in a Note

Primary Actor: User

Preconditions:

- A note is opened

Trigger:

- User adds/edits table rows

Main Success Scenario:

1. User adds or modifies an entry
2. System validates date & price
3. On exit or save, shows confirmation dialog
4. System updates saved content

UC7: Calculate Total Expense

Primary Actor: User

Preconditions:

- Note has at least one price entry

Trigger:

→ User clicks “Calculate Total” button

Main Success Scenario:

1. System sums all prices in note
2. Displays total at bottom

UC8: Set Reminder/Alert

Primary Actor: User

Secondary Actor: Reminder System

Preconditions:

→ User is logged in

Trigger:

→ User clicks “Marking” or “Add Reminder”

Main Success Scenario:

1. User selects date and adds topic (optional description)
2. System stores reminder
3. On date match, System triggers in-app notification

UC9: View Notifications

Primary Actor: User

Secondary Actor: Notification System

Preconditions:

→ Reminders/alerts exist

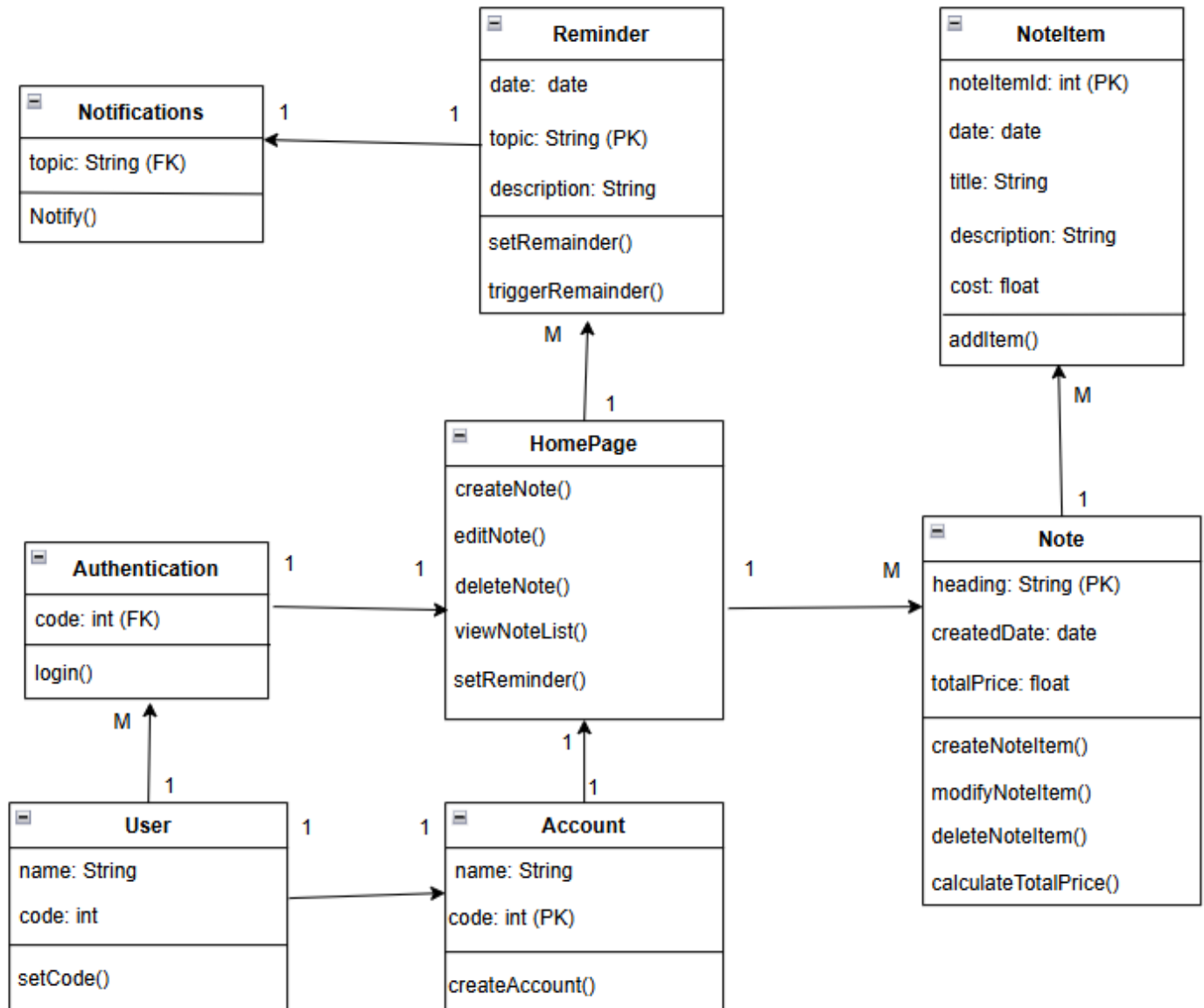
Trigger:

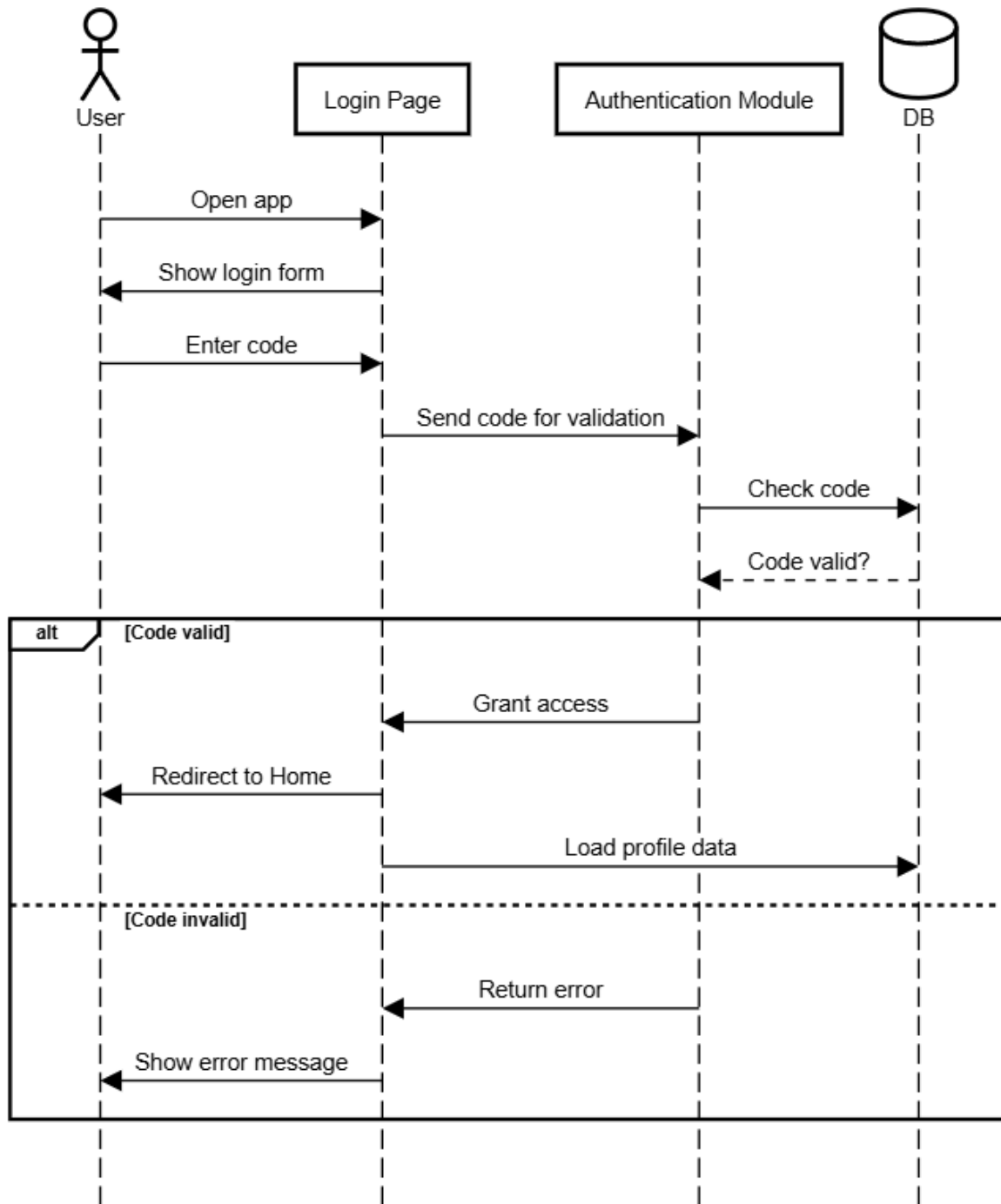
→ App is opened or user clicks on “Notifications”

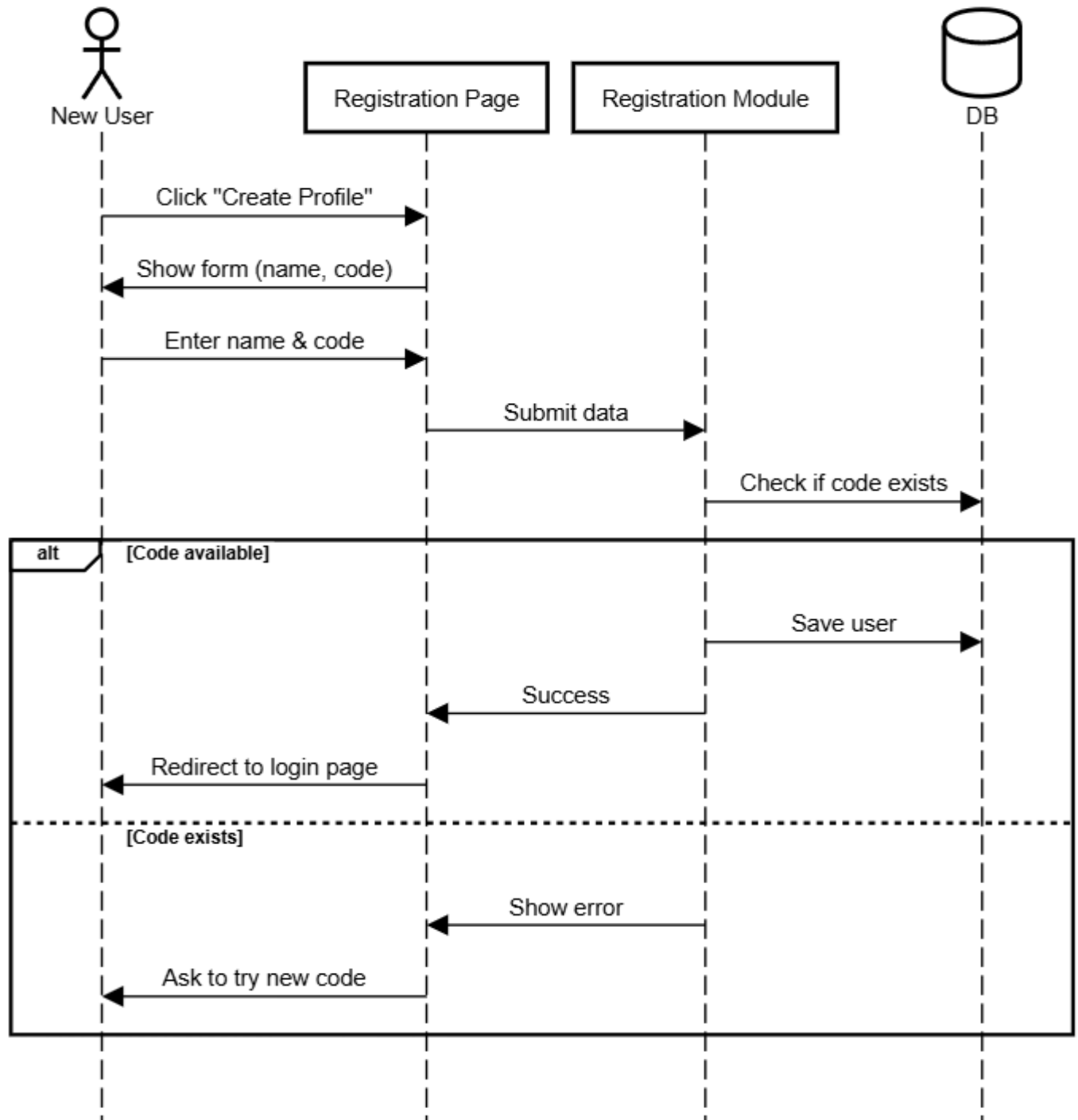
Main Success Scenario:

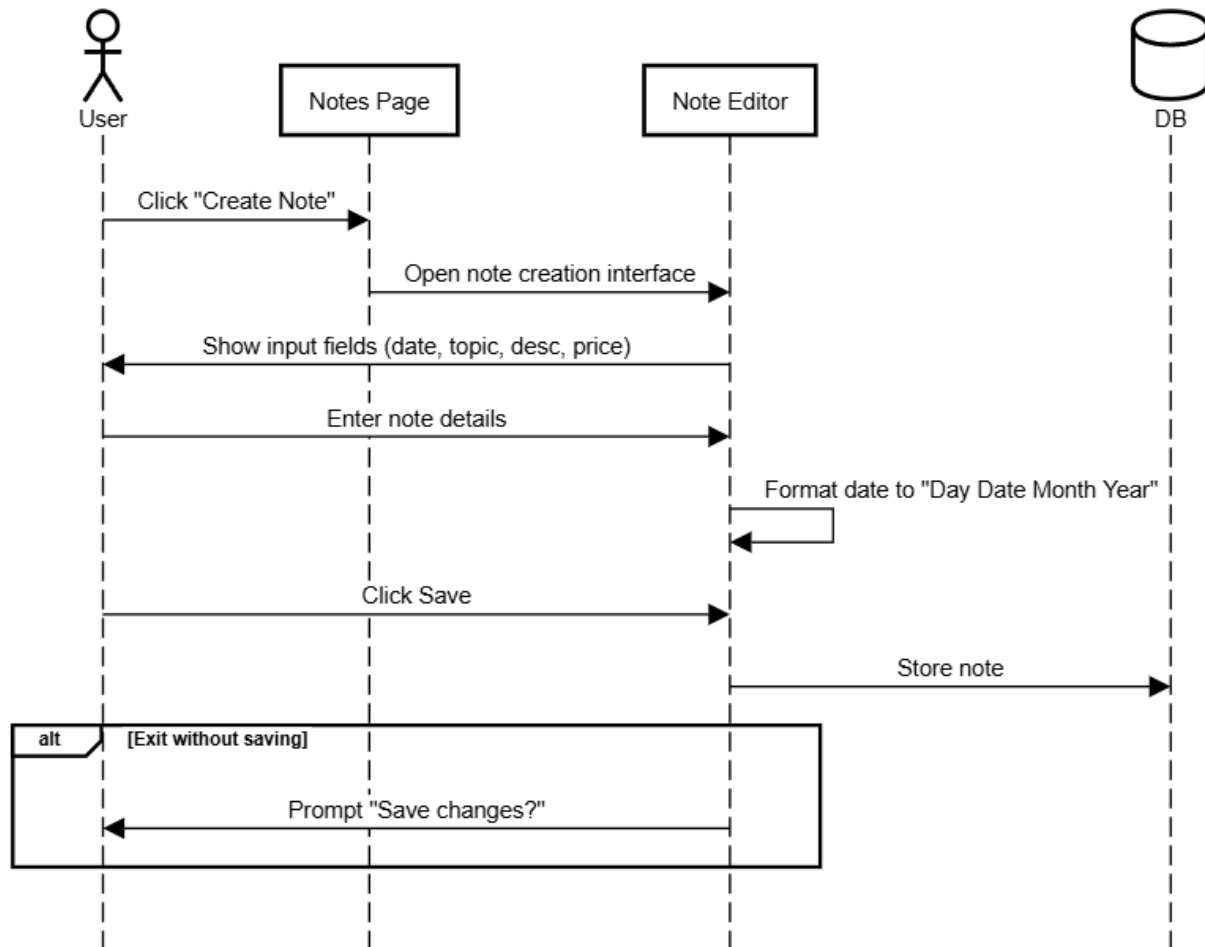
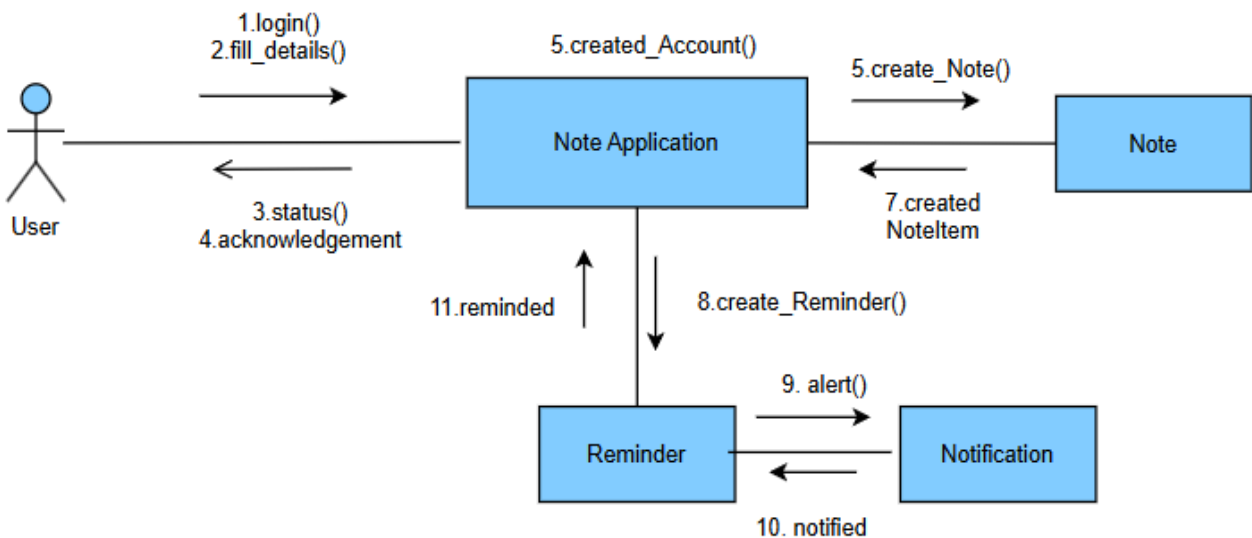
1. System displays due/past notifications
2. User acknowledges or dismisses alerts

Class Diagram

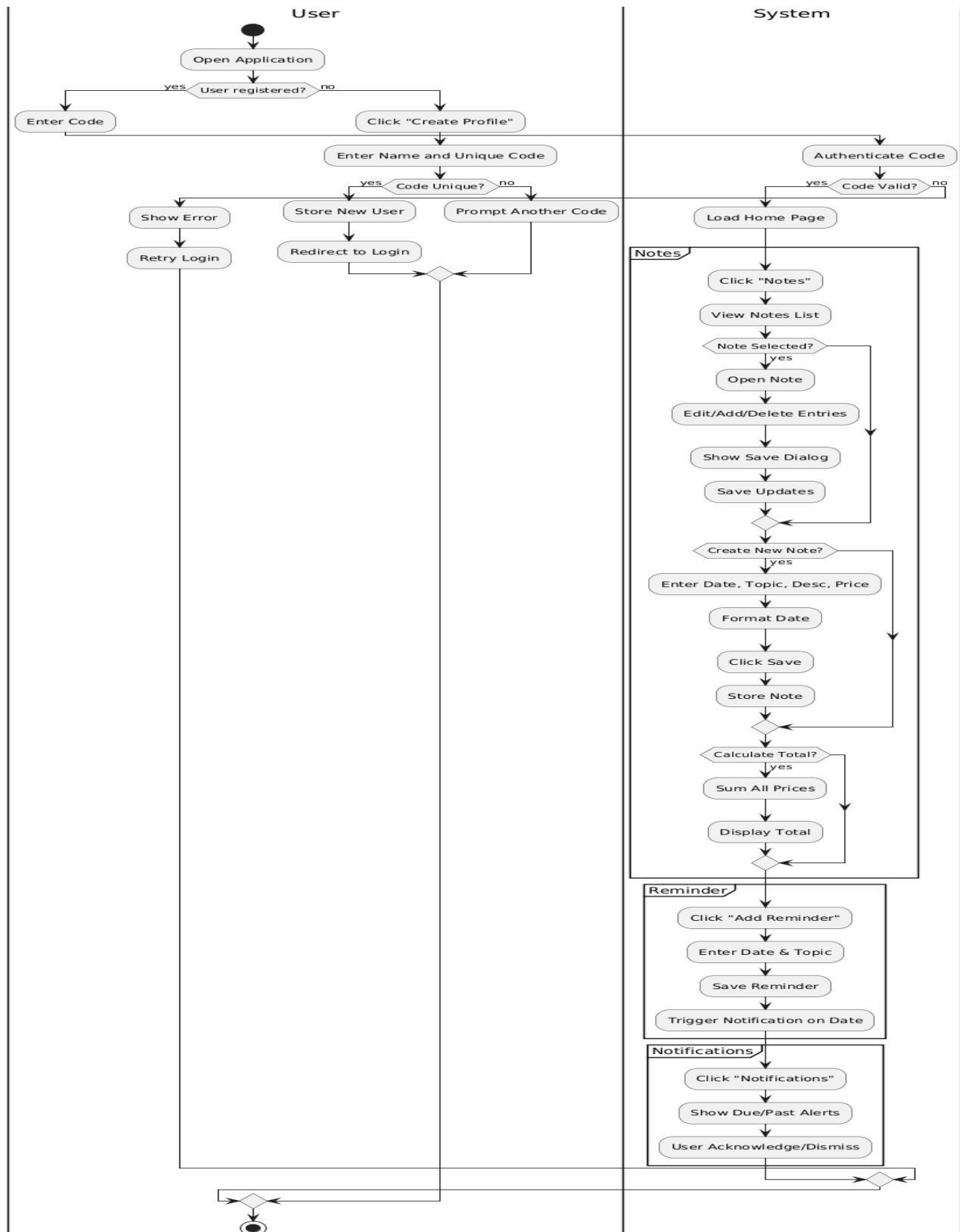


SEQUENCE DIAGRAM 1:

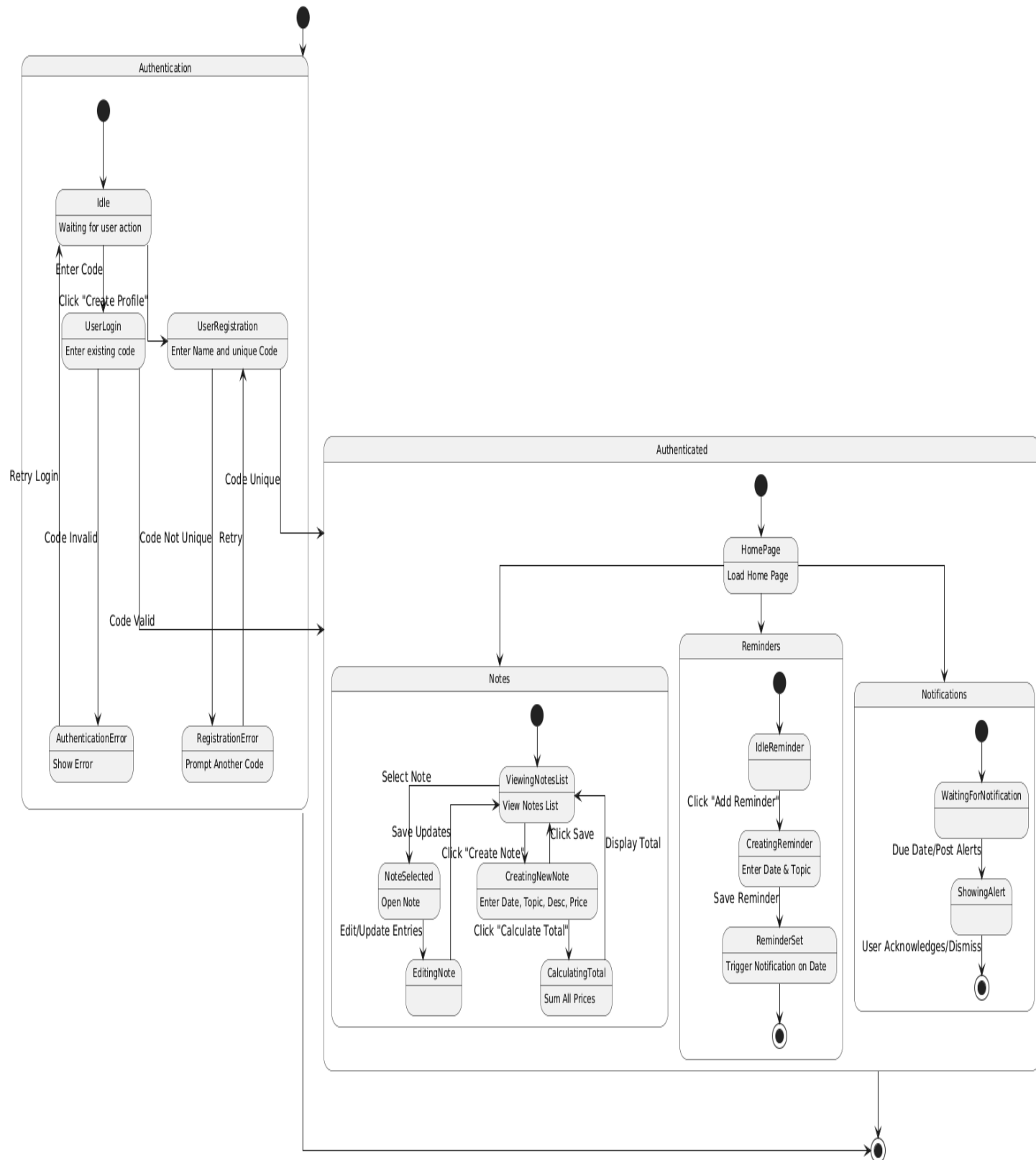
SEQUENCE DIAGRAM 2:

SEQUENCE DIAGRAM 3:**COLLABORATION DIAGRAM:**

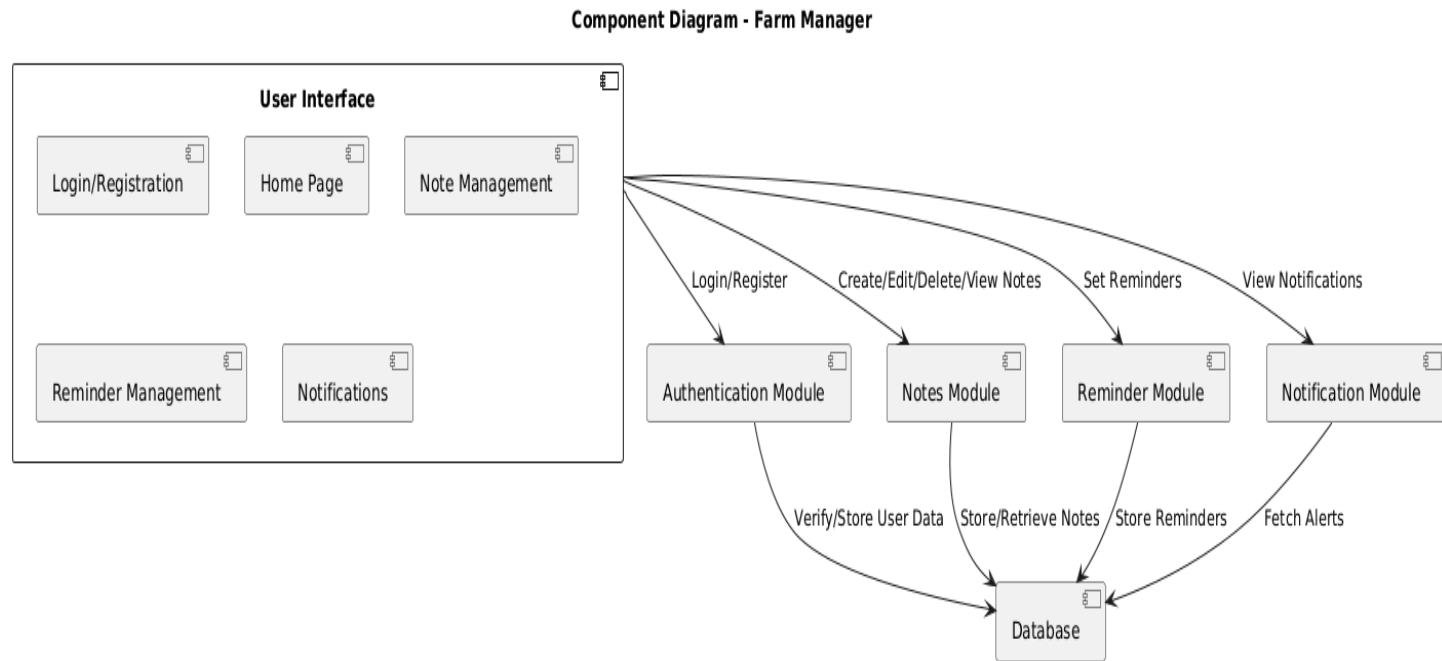
ACTIVITY DIAGRAM:



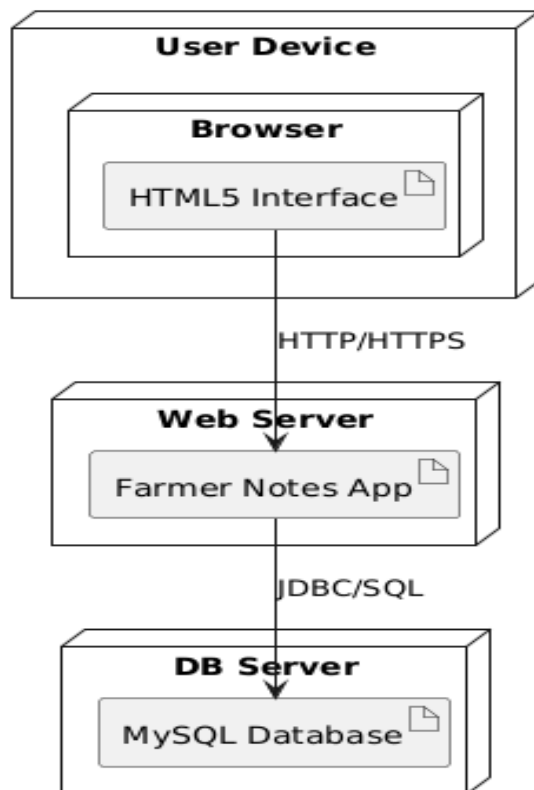
STATE DIAGRAM:



COMPONENT DIAGRAM:



DEPLOYMENT DIAGRAM:



5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Load time for each page must not exceed 2 seconds on standard broadband
- System supports simultaneous usage by 10+ users without performance degradation

5.2 Safety Requirements

- Graceful handling of unexpected shutdowns with minimal data loss
- Robust error handling for all user input

5.3 Security Requirements

- Secure storage and hashing of user codes
- No sensitive data stored in plaintext or transmitted insecurely

5.4 Software Quality Attributes

- High usability: minimal screens and steps per workflow
- Reliability: consistent data saving and retrieval
- Maintainability: modular Flask codebase and clear documentation

5.5 Business Rules

- Only registered users can create, edit, or delete notes
- Users cannot access or modify other users' records

6. Other Requirements

- The software must be translatable/localizable for different languages/regions
- Data export (CSV/Excel) feature for records (future enhancement)
- Compliance with basic privacy protection regulations

Appendix A: Glossary

- **Note:** Entry containing date, topic, description, price
- **Reminder:** Date-based alert for a future event
- **Authentication Code:** Unique user credential for login

Appendix B: Analysis Models

- Include diagrams: Use case, class, sequence, activity as required for documentation or demonstration. (Refer to your diagrams or draw as needed.)

Appendix C: To Be Determined List

- Specify any requirements or features left undefined for future updates.

CODE MODULES

APP.PY

```
from flask import Flask, render_template, request, redirect, url_for, session, flash, jsonify

import sqlite3

from datetime import datetime

app = Flask(__name__)

app.secret_key = 'your_secret_key'

# Database setup

conn = sqlite3.connect('notes.db')

c = conn.cursor()

c.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, name TEXT, code TEXT)")

c.execute("CREATE TABLE IF NOT EXISTS notes (id INTEGER PRIMARY KEY, user_id INTEGER, title TEXT, date TEXT)")

c.execute("CREATE TABLE IF NOT EXISTS note_entries (id INTEGER PRIMARY KEY, note_id INTEGER, date TEXT, topic TEXT, description TEXT, price REAL)")

c.execute("CREATE TABLE IF NOT EXISTS reminders (id INTEGER PRIMARY KEY, user_id INTEGER, topic TEXT, reminder_date TEXT, created_at TEXT)")

c.execute("CREATE TABLE IF NOT EXISTS notifications (id INTEGER PRIMARY KEY, user_id INTEGER, message TEXT, type TEXT, created_at TEXT, is_read INTEGER DEFAULT 0)")

conn.commit()

conn.close()

@app.route('/')

def index():

    return redirect(url_for('login'))

@app.route('/login', methods=['GET', 'POST'])

def login():
```

```
if request.method == 'POST':
    code = request.form['code']
    conn = sqlite3.connect('notes.db')
    c = conn.cursor()
    c.execute('SELECT * FROM users WHERE code=?', (code,))
    user = c.fetchone()
    conn.close()
    if user:
        session['user_id'] = user[0]
        return redirect(url_for('home'))
    flash("Invalid code")
return render_template('login.html')

def add_notification(user_id, message, notification_type):
    conn = sqlite3.connect('notes.db')
    c = conn.cursor()
    created_at = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    c.execute('INSERT INTO notifications (user_id, message, type, created_at) VALUES (?, ?, ?, ?)',
              (user_id, message, notification_type, created_at))
    conn.commit()
    conn.close()

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        name = request.form['name']
        code = request.form['code']
        conn = sqlite3.connect('notes.db')
        c = conn.cursor()
        c.execute('INSERT INTO users (name, code) VALUES (?, ?)', (name, code))
        conn.commit()
        conn.close()
        flash("Account created. Login now.")
        add_notification(c.lastrowid, "Welcome to Farm Manager! Your account has been successfully
        created.", "alert")
        add_notification(c.lastrowid, "Don't forget to check the weather forecast for optimal planting
        conditions today!", "weather")
```

```
    add_notification(c.lastrowid, "Reminder: Your irrigation system needs maintenance next week.", "farming")
```

```
    add_notification(c.lastrowid, "New research indicates advanced fertilizers can boost your yield by 15% this season!", "farming")
```

```
    add_notification(c.lastrowid, "Government announces new agricultural subsidies. Check details on our schemes page!", "alert")
```

```
    return redirect(url_for('login'))
```

```
    return render_template('register.html')
```

```
@app.route('/home')
```

```
def home():
```

```
    if 'user_id' not in session:
```

```
        return redirect(url_for('login'))
```

```
    conn = sqlite3.connect('notes.db')
```

```
    c = conn.cursor()
```

```
    today = datetime.now().strftime("%Y-%m-%d")
```

```
    c.execute('SELECT topic FROM reminders WHERE user_id=? AND reminder_date=?',  
(session['user_id'], today))
```

```
    reminders = c.fetchall()
```

```
    conn.close()
```

```
    return render_template('home.html', reminders=reminders)
```

```
@app.route('/notifications')
```

```
def get_notifications():
```

```
    if 'user_id' not in session:
```

```
        return redirect(url_for('login'))
```

```
    user_id = session['user_id']
```

```
    conn = sqlite3.connect('notes.db')
```

```
    c = conn.cursor()
```

```
    c.execute('SELECT id, message, type, created_at, is_read FROM notifications WHERE  
user_id=? ORDER BY created_at DESC', (user_id,))
```

```
    notifications = c.fetchall()
```

```
    conn.close()
```

```
    notifications_list = []
```

```
    for notif in notifications:
```

```
        notifications_list.append({
```

```
            'id': notif[0],
```

```
        'message': notif[1],
        'type': notif[2],
        'created_at': notif[3],
        'is_read': bool(notif[4])
    })
    return jsonify(notifications_list)

@app.route('/notifications_page')
def notifications_page():
    if 'user_id' not in session:
        return redirect(url_for('login'))
    user_id = session['user_id']
    conn = sqlite3.connect('notes.db')
    c = conn.cursor()
    c.execute('SELECT id, message, type, created_at, is_read FROM notifications WHERE
user_id=? ORDER BY created_at DESC', (user_id,))
    notifications = c.fetchall()
    conn.close()
    notifications_list = []
    for notif in notifications:
        notifications_list.append({
            'id': notif[0],
            'message': notif[1],
            'type': notif[2],
            'created_at': notif[3],
            'is_read': bool(notif[4])
        })

    return render_template('notifications.html', notifications=notifications_list)
@app.route('/notifications/mark_read/<int:notification_id>', methods=['POST'])
def mark_notification_read(notification_id):
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': 'Unauthorized'}), 401
    conn = sqlite3.connect('notes.db')
    c = conn.cursor()
```



```
c.execute('UPDATE notifications SET is_read=1 WHERE id=? AND user_id=?',
(notification_id, session['user_id']))
conn.commit()
conn.close()
return jsonify({'success': True, 'message': 'Notification marked as read'})
@app.route('/notifications/mark_all_read', methods=['POST'])
def mark_all_notifications_read():
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': 'Unauthorized'}), 401
    user_id = session['user_id']
    conn = sqlite3.connect('notes.db')
    c = conn.cursor()
    c.execute('UPDATE notifications SET is_read=1 WHERE user_id=?', (user_id,))
    return jsonify({'success': True, 'message': 'All notifications marked as read'})
@app.route('/notes')
def notes():
    if 'user_id' not in session:
        return redirect(url_for('login'))
    conn = sqlite3.connect('notes.db')
    c = conn.cursor()
    c.execute('SELECT * FROM notes WHERE user_id=?', (session['user_id'],))
    notes = c.fetchall()
    conn.close()
    return render_template('notes.html', notes=notes)
@app.route('/create_note', methods=['POST'])
def create_note():
    title = request.form['title']
    date = datetime.now().strftime("%A, %d %B %Y")
    conn = sqlite3.connect('notes.db')
    c = conn.cursor()
    c.execute('INSERT INTO notes (user_id, title, date) VALUES (?, ?, ?)', (session['user_id'], title,
date))
    note_id = c.lastrowid
    flash("New note created successfully! Start adding your farming entries.")
    return redirect(url_for('view_note', note_id=note_id))
```

```

@app.route('/note/<int:note_id>', methods=['GET', 'POST'])
def view_note(note_id):
    now = datetime.now()
    conn = sqlite3.connect('notes.db')
    c = conn.cursor()
    if request.method == 'POST':
        if 'day' in request.form and 'month' in request.form and 'year' in request.form:
            day = int(request.form['day'])
            year = int(request.form['year'])
            date = datetime(year, month, day).strftime("%A, %d %B %Y")
        else:
            date = datetime.now().strftime("%A, %d %B %Y")
        topic = request.form['topic']
        desc = request.form['desc']
        price = float(request.form['price'] or 0)
        c.execute('INSERT INTO note_entries (note_id, date, topic, description, price) VALUES (?, ?, ?, ?, ?)',
            (note_id, date, topic, desc, price))
        conn.commit()
        flash("Entry added. Remember to save before exiting.")
        c.execute('SELECT title FROM notes WHERE id=?', (note_id,))
        title = c.fetchone()[0]
        c.execute('SELECT * FROM note_entries WHERE note_id=?', (note_id,))
        entries = c.fetchall()
        total = sum([entry[5] for entry in entries])
        conn.close()
        return render_template(
            current_day=now.day,
            current_month=now.month,
            current_year=now.year
        )
@app.route('/bulk_delete_entries/<int:note_id>', methods=['POST'])
def bulk_delete_entries(note_id):
    entry_ids = request.form.getlist('entry_ids')
    if entry_ids:

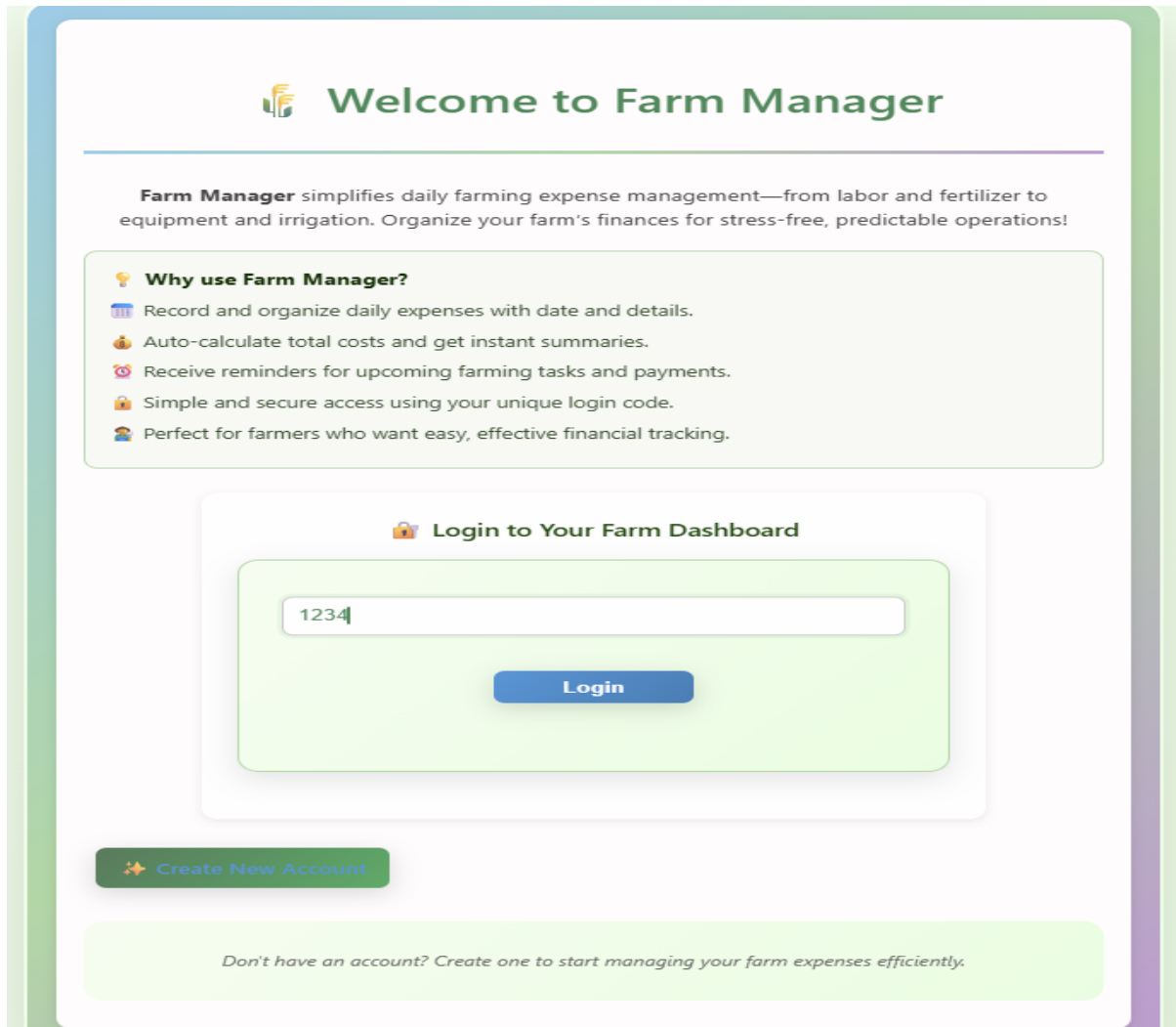
```

```

conn = sqlite3.connect('notes.db')
c = conn.cursor()
# Delete multiple entries at once
placeholders = ','.join('? ' * len(entry_ids))
c.execute(f'DELETE FROM note_entries WHERE id IN ({placeholders})', entry_ids)
conn.commit()
flash(f'{len(entry_ids)} entries deleted successfully.')
else:
    flash("No entries selected for deletion.")
    return redirect(url_for('view_note', note_id=note_id))
@app.route('/logout')
def logout():
    session.clear()
@app.route('/set_reminder', methods=['GET', 'POST'])
def set_reminder():
    if 'user_id' not in session:
        return redirect(url_for('login'))
    if request.method == 'POST':
        topic = request.form['topic']
        reminder_date_str = request.form['reminder_date']
        created_at = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        conn = sqlite3.connect('notes.db')
        c = conn.cursor()
        c.execute('INSERT INTO reminders (user_id, topic, reminder_date, created_at) VALUES (?, ?, ?)',
            (session['user_id'], topic, reminder_date_str, created_at))
        conn.commit()
        conn.close()
        flash("Reminder set successfully!")
        add_notification(session['user_id'], f'New reminder set for: {topic} on {reminder_date_str}.',
            "farming")
        return redirect(url_for('home'))
    return render_template('reminders.html')
if __name__ == '__main__':
    app.run(debug=True)

```

LOGIN PAGE:



The login page features a light green background with a white central content area. At the top, a logo of three stylized leaves is followed by the heading "Welcome to Farm Manager". Below this, a paragraph describes the app's purpose: "Farm Manager simplifies daily farming expense management—from labor and fertilizer to equipment and irrigation. Organize your farm's finances for stress-free, predictable operations!". A section titled "Why use Farm Manager?" lists five benefits with icons: record expenses, auto-calculate costs, receive reminders, secure access, and ease of use. The main login section, "Login to Your Farm Dashboard", contains a text input field with "1234" and a blue "Login" button. A green button labeled "Create New Account" is positioned below. At the bottom, a light green box contains the text: "Don't have an account? Create one to start managing your farm expenses efficiently."

Welcome to Farm Manager

Farm Manager simplifies daily farming expense management—from labor and fertilizer to equipment and irrigation. Organize your farm's finances for stress-free, predictable operations!

Why use Farm Manager?

- Record and organize daily expenses with date and details.
- Auto-calculate total costs and get instant summaries.
- Receive reminders for upcoming farming tasks and payments.
- Simple and secure access using your unique login code.
- Perfect for farmers who want easy, effective financial tracking.

Login to Your Farm Dashboard

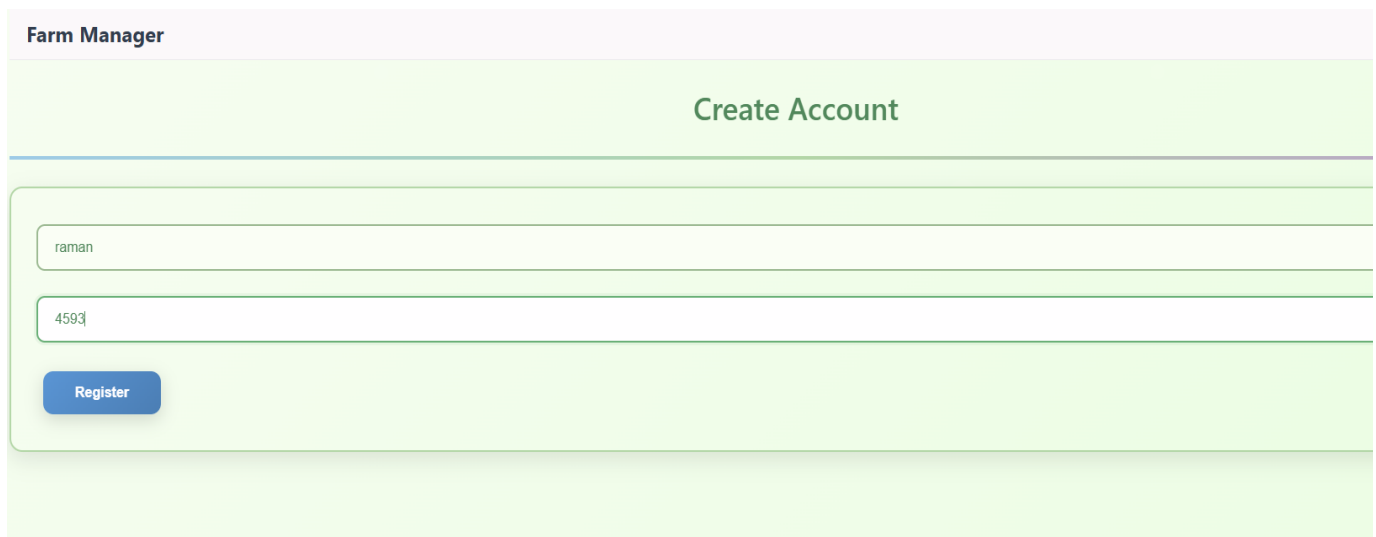
1234

Login

Create New Account

Don't have an account? Create one to start managing your farm expenses efficiently.

REGISTRATOIN PAGE



The registration page has a light green background. A header bar at the top contains the text "Farm Manager" on the left and "Create Account" on the right. Below the header, there are two text input fields. The first field contains the text "raman" and the second field contains "4593". A blue "Register" button is located below the input fields.

Farm Manager

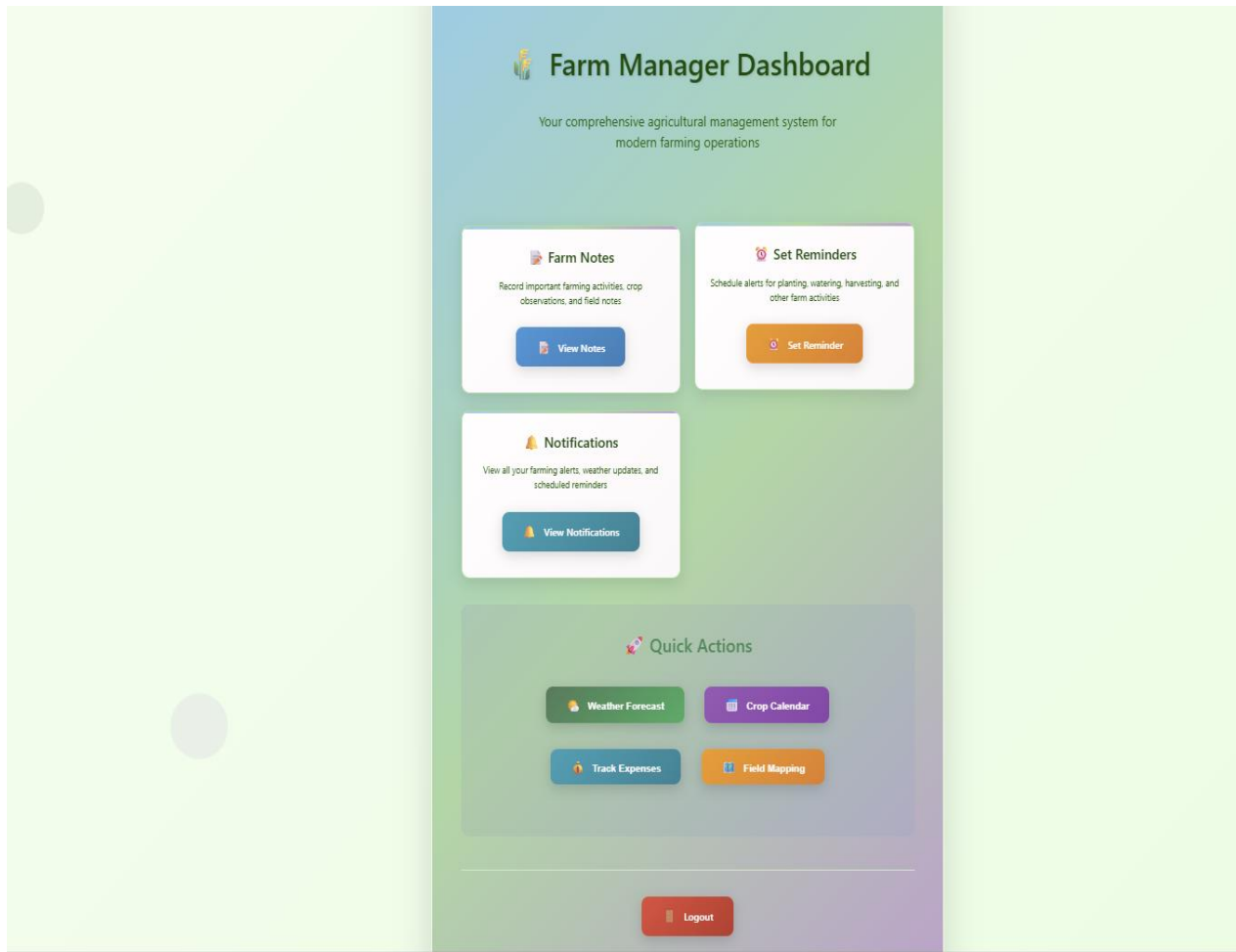
Create Account

raman

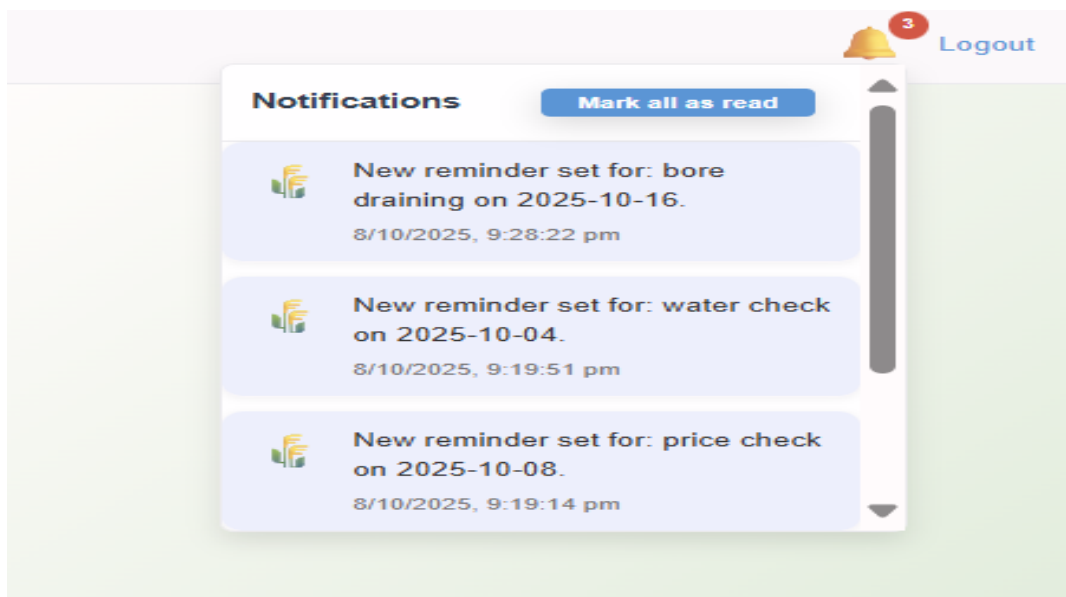
4593

Register

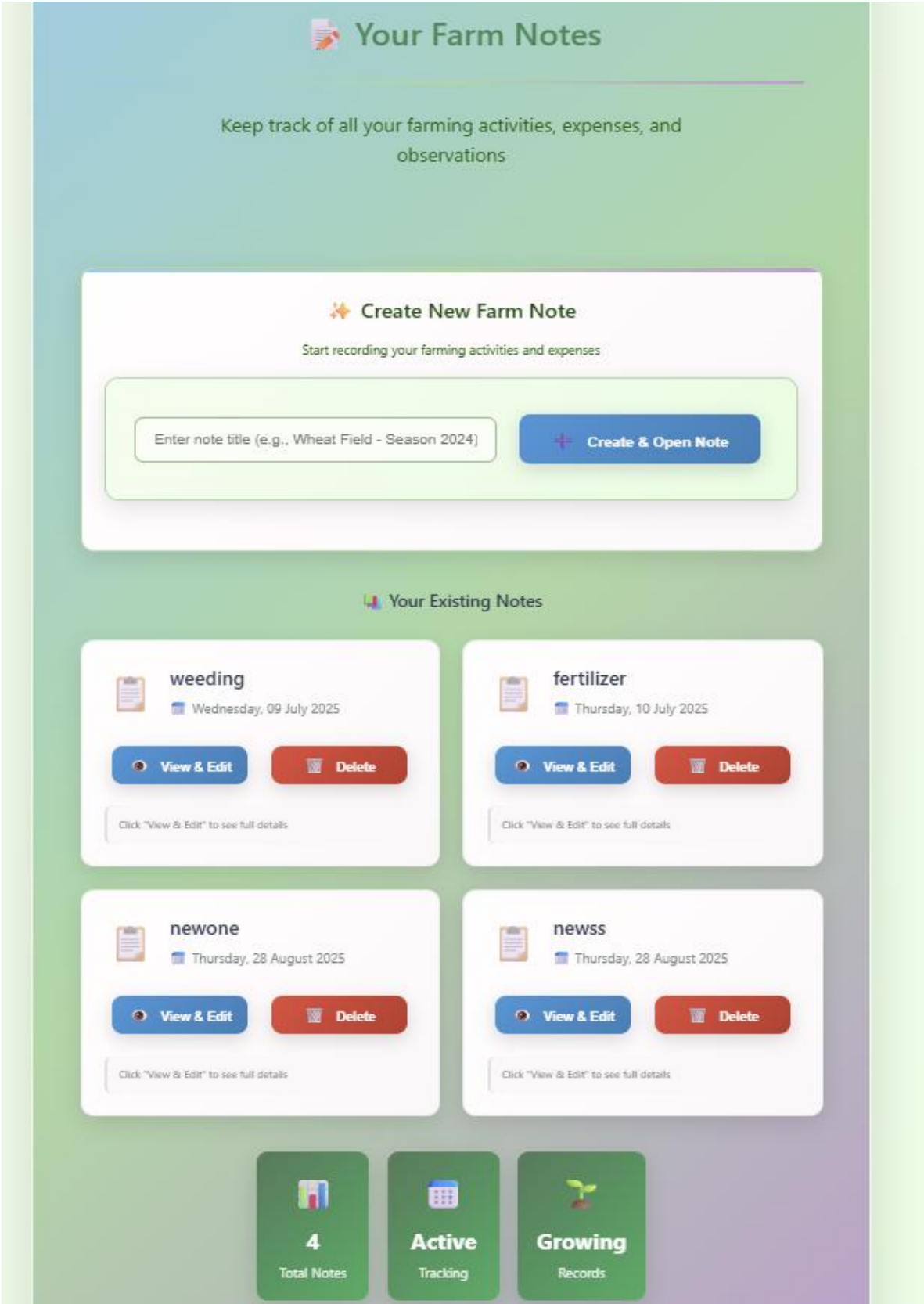
DASHBOARD PAGE



NOTIFICATION ALERT



NOTES PAGE



VIEW NOTES PAGE

Farm Manager

Entry added. Remember to save before exiting.

Back to Notes

fertilizer

8

▼

October

▼

2025

▼

Topic

Description

Amount

Add Entry

Delete

S.NO	DATE	TOPIC	DESCRIPTION	PRICE
1	Wednesday, 08 October 2025	potash	5 KG	₹580.0
2	Saturday, 11 October 2025	NPK	3 KG	₹276.0
3	Thursday, 16 October 2025	Complex	6 KG	₹1354.0

Total: ₹2210.0

DELETE PAGE

Farm Manager

127.0.0.1:5000 says
Are you sure you want to delete 2 selected entries? This action cannot be undone.

Logout

Back to Notes

fertilizer

8

October

2025

Topic

Description

Amount

Add Entry

Cancel

Delete Selected

	S.NO	DATE	TOPIC	DESCRIPTION	PRICE
☒	1	Wednesday, 08 October 2025	potash	5 KG	₹580.0
☒	2	Saturday, 11 October 2025	NPK	3 KG	₹276.0
☐	3	Thursday, 16 October 2025	Complex	6 KG	₹1354.0

NOTIFICATION PAGE

Farm Manager

Logout

Back to Home

Your Notifications

New reminder set for: bore draining on 2025-10-16.
2025-10-08 21:28:22

New reminder set for: water check on 2025-10-04.
2025-10-08 21:19:51

New reminder set for: price check on 2025-10-08.
2025-10-08 21:19:14

New reminder set for: testing on 2025-09-24.
2025-09-24 21:41:11

Mark All as Read

REMINDER PAGE

The screenshot shows the 'Set a New Reminder' page in the Farm Manager application. The page has a light green background. At the top left, there is a 'Farm Manager' header. At the top right, there is a 'Logout' button with a bell icon. Below the header, there is a 'Back to Dashboard' button. The main content area is a large green card with a blue header that says 'Set a New Reminder' with a bell icon. Below the header, there is a sub-header 'Create Reminder' and a prompt 'Enter the details for your new reminder'. There are two input fields: the first is labeled 'bore draining' and the second is labeled '16-18-2025'. Below the input fields is a blue button labeled 'Set Reminder'.

REMINDER ALERT

The screenshot shows a 'Reminder Alert' dialog box. The dialog has a pink background. At the top left, there is a bell icon and the text 'Reminder!'. At the top right, there is a close button (X). Below the title, there is the text 'price check'. At the bottom, there is a button labeled 'Notify Later Today'.